

Opposition Agents Playing Magic: The Gathering

Knowledge-Graph-Grounded JEPA World Models and Active-Inference LLM Agents

Felix Neubürger¹ and Jonas Neubürger¹

¹Independent Research , github.com/cuteredpwnda/opposition-agents-playing-mtg

April 2026

Abstract

Magic: The Gathering (MTG) is, by several measures, the most combinatorially complex commercial card game ever published: more than 28,000 unique cards, a 260-page rule book, partial observability over 50–90 hidden cards, an unbounded action space, a turn-based stack with priority, and a state space conservatively estimated above 10^{800} . We present OPPOSITION AGENTS PLAYING MTG, an open-source research framework that combines (i) a from-scratch implementation of the Comprehensive Rules game engine, (ii) a Neo4j knowledge graph backed by an OWL 2 ontology of cards, mechanics, archetypes and combos, (iii) a four-component world model in the V+M+C tradition of [Ha & Schmidhuber \(2018\)](#) extended with a Joint-Embedding Predictive Architecture (JEPA) head in the spirit of [LeCun \(2022\)](#), and (iv) Active-Inference LLM agents that fuse symbolic graph queries, latent imagination and free-energy-minimising action selection. We describe the system end-to-end, justify the design choices against the structure of the game, and report on a champion-vs-challenger self-play promotion loop with ELO tracking. The code, ontology, and this report are released under MIT.

Keywords: world models, JEPA, knowledge graphs, active inference, LLM agents, self-play, neuro-symbolic reasoning, Magic: The Gathering.

1 Introduction

Reinforcement-learning research on games has historically progressed through a sequence of increasingly hard environments: backgammon ([Tesauro, 1995](#)), chess and Go ([Silver et al., 2017, 2018](#)), StarCraft II ([Vinyals et al., 2019](#)), and no-limit poker ([Brown & Sandholm, 2018](#)). Each step pushed the community to invent new tools — temporal-difference learning, deep value networks, self-play, counterfactual regret minimisation. *Magic: The Gathering* sits beyond the union of these challenges (Table 1): its action space is unbounded, its rules are open-textured natural language printed on cards (the rule book itself defers card text), its hidden information includes the entire opponent’s deck composition, and its mechanics are extended every quarter by the publisher. A single MTG card can override the rules of the game, and a single decision in combat may resolve dozens of triggered abilities in a player-chosen order.

A purely model-free deep-RL approach is therefore unattractive: the action space cannot be enumerated as a fixed-size logits vector, the observation space is symbolic and irregular, and every new set introduces previously-unseen card text whose semantics must be parsed at run time. Conversely, a purely symbolic LLM-prompting approach struggles with rules-correctness, hallucinated cards, and the long-horizon credit-assignment that competitive play requires.

Our position is that the structure of MTG demands a *neuro-symbolic* stack: a verified rules engine to keep play legal, a knowledge graph to make card semantics queryable, a learned world

Table 1: Approximate combinatorial complexity comparison.

	Chess	Go	No-limit Poker (HU)	MTG
State space	10^{47}	10^{170}	10^{160}	$> 10^{800}$
Branching factor	~ 35	~ 250	~ 10	0–100+ (unbounded)
Hidden information	none	none	2 cards	50–90+ cards
Card / piece pool	6 types	1 type	52 cards	$> 28,000$ unique
Rule text length	$\sim 10p$	$\sim 5p$	$\sim 10p$	$\sim 260p$ + per-card
Rules update frequency	decades	decades	decades	quarterly

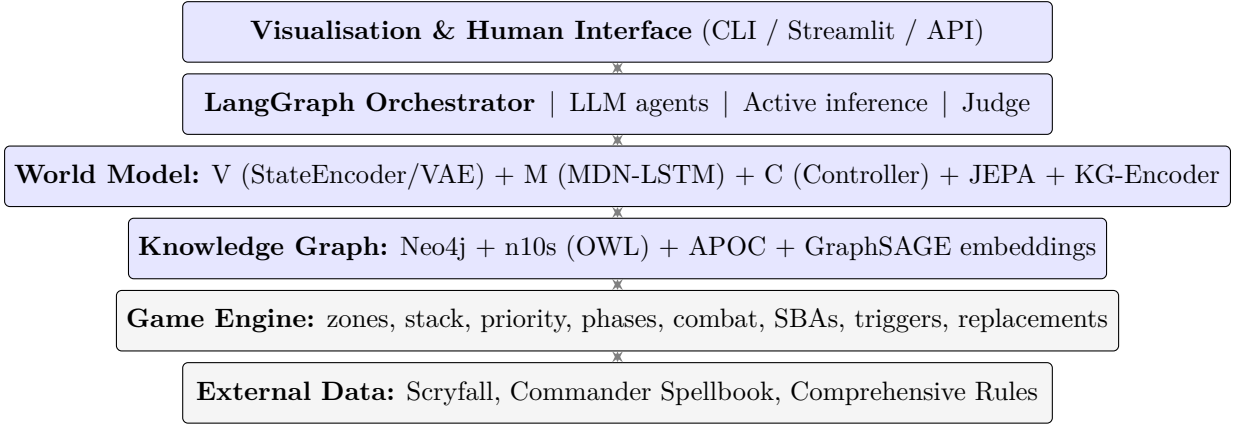


Figure 1: Five-layer architecture of the framework. Symbolic verification (engine, KG) sits below latent imagination (world model) which is wrapped by an LLM-driven orchestrator.

model to enable imagination and planning over the resulting compressed state, and an LLM agent that orchestrates these components under an active-inference objective. Figure 1 sketches the resulting five-layer architecture.

Contributions. This work makes four contributions.

1. A *from-scratch* Comprehensive-Rules MTG engine in Python with stack/priority, layered continuous effects, replacement effects, triggers and state-based actions, exposing a deterministic step API suitable for batched self-play.
2. An *ontology-grounded knowledge graph* of cards, mechanics, archetypes and combos, imported into Neo4j via the **n10s** OWL importer and made queryable from agents through a thin GraphRAG layer.
3. A $V+M+C+JEPA$ world model where the state encoder fuses tokenised game state with a graph-context vector from the KG, the dynamics model is an MDN-LSTM over the latent, and a JEPA head provides a contrastive predictor for surprise scoring and dream rollouts.
4. An *active-inference LLM-fusion agent* that chooses actions by minimising expected free energy combining pragmatic value, epistemic value, and a calibrated LLM prior; a self-play training loop with ELO ranking, opponent-pool sampling and champion-vs-challenger promotion is provided.

The framework is released as open source¹ and the codebase is the canonical artifact accompanying this report.

¹<https://github.com/cuteredpwnda/opposition-agents-playing-mtg>

2 Related Work

World models. Ha & Schmidhuber (2018) introduced the V+M+C decomposition — a vision-VAE, a recurrent dynamics model with mixture-density output, and a tiny linear/MLP controller trained in the dream of M. Successors include Dreamer (Hafner et al., 2020, 2023) and the Schmidhuber-line of self-referential systems (Schmidhuber, 2015; Irie et al., 2022). We adopt the V+M+C skeleton but (i) replace the convolutional vision encoder with a Set-Transformer over symbolic game tokens, and (ii) attach a JEPA head (LeCun, 2022; Assran et al., 2023) that predicts a target encoding rather than a pixel-level reconstruction — a closer fit to MTG, where “imagination” should preserve abstract game state, not card art.

Active inference. The Free-Energy Principle of Friston (2010) and its discrete-state operationalisation (Sajid et al., 2021) provide a principled objective for action selection under uncertainty: minimise expected free energy, which decomposes into a pragmatic (reward-seeking) and an epistemic (information-seeking) term. We use this decomposition to balance “play to win” against “probe the opponent” decisions during the early game.

Knowledge graphs and GraphRAG. Neo4j with the `n10s` plugin enables direct ingestion of OWL ontologies as labeled property graphs (Neo4j Labs, n.d.); the APOC library adds graph algorithms; GraphSAGE (Hamilton et al., 2017) provides inductive node embeddings. GraphRAG (Edge et al., 2024) has recently emerged as a complement to vector RAG for queries that benefit from explicit relational structure. MTG is a textbook GraphRAG case: “find me a two-card combo using only cards in my library that survive Counterspell” is a multi-hop graph query, not a similarity search.

MTG and other complex card games. Earlier MTG-AI work has used hand-crafted heuristics (Cowling et al., 2012), MCTS, and small policy nets on restricted card pools (e.g., `open-mtg`). On the rules side, `Forge`, `XMage` and `Mage` provide JVM-based reference implementations. To our knowledge, no prior public system combines a full Comprehensive-Rules engine with a learned world model and a graph-grounded LLM controller. Moravčík et al. (2017) (poker), Vinyals et al. (2019) (StarCraft) and Schrittwieser et al. (2020) (MuZero) are spiritual antecedents at the algorithmic level.

Ontologies for games. Game ontologies have been explored for serious games and educational metadata; for trading-card games specifically, public OWL ontologies are sparse. Our ontology (`data/ontology/mtg-ontology-v1.1.owl`) draws on the practices surveyed in Noy & McGuinness (2001) and the OntologyExtender workflow of Sarker et al. (2024), and is validated by SHACL shapes (Knublauch & Kontokostas, 2017).

3 System Architecture

3.1 Game engine

The engine implements the Comprehensive Rules subset relevant for two-player Standard plus the structural extensions for multiplayer Commander. Two parallel orchestrators sit on top of the same dataclasses: a synchronous `GameSimulator` (`src/engine/game_simulator.py`) used in tournaments and tests, and an asynchronous `GameRunner` (`src/orchestrator/game_runner.py`) used in self-play data collection. Both implement the same end-of-game rules. Core modules:

- `game_state`: immutable-by-convention dataclasses for `GameState`, `PlayerState` (carrying `max_hand_size` and `mulligans_taken`), and `CardInstance`;

- **zones, stack, phases, priority_loop**: APNAP priority and the seven-step turn structure;
- **rules_engine**: legal-action generation given current priority and phase;
- **state_based_actions** (CR 704), **triggered_abilities** (CR 603), **replacement_effects** (CR 614), **continuous_effects** (CR 613 layer system);
- **combat, mana, keywords, abilities**: per-mechanic logic;
- **London mulligan** at game setup: draw 7, optionally re-mulligan up to a configurable cap, then bottom cards equal to the number of mulligans taken (CR 103.5);
- **Cleanup-phase discard** down to `PlayerState.max_hand_size` (default 7), which had been a long-standing gap in earlier engine snapshots (CR 514);
- **Deterministic empty-library loss**: drawing from an empty library ends the game with that player losing (CR 104.3c / 704.5b), in both the sync simulator and the async runner;
- **Deterministic timeout tie-breaker**: instead of auto-draw on hitting the configured turn limit, the leader on the lexicographic tuple (life total, permanents in play, hand size, library size) wins; only true ties are recorded as draws.

The engine is pure-Python, deterministic given an RNG seed, and has no LLM dependency, which makes it the ground truth that all agents play against.

3.2 Knowledge graph

Cards are imported from Scryfall and reified as `(:Card)` nodes carrying mana cost, type line, oracle text, colour identity, and a SBERT embedding of the oracle text. Mechanics, abilities, archetypes and combos become first-class nodes connected by typed edges (`HAS_KEYWORD`, `ENABLES_COMBO`, `ARCHETYPE_OF`, ...). The OWL ontology (v1.1) declares the schema; `n10s` imports it; `pyshacl` validates it; `APOC` provides PageRank, community detection and shortest-path queries used in agent reasoning. ?? 1 shows a typical multi-hop query used by the combo-detector agent.

Listing 1: Two-card combo query.

```
MATCH (c1:Card)-[:ENABLES_COMBO]->(combo:Combo)<-[:ENABLES_COMBO]-(c2:Card)
WHERE c1.name IN $library AND c2.name IN $library AND c1 <> c2
RETURN combo.name, combo.win_condition, c1.name, c2.name
ORDER BY combo.cmc_total ASC LIMIT 10;
```

3.3 World model

Let $s_t \in \mathcal{S}$ be the game state at decision step t and $g \in \mathcal{G}$ the player’s KG context (deck, archetype tag, known opponent information). The world model decomposes as:

$$z_t = \text{Enc}_V(s_t) \quad (\text{state encoder, Set-Transformer + VAE}) \quad (1)$$

$$g_t = \text{Enc}_{\text{KG}}(g, s_t) \quad (\text{KG context encoder}) \quad (2)$$

$$h_t, \pi_t^M = \text{LSTM}_M([z_t; g_t], h_{t-1}) \quad (\text{MDN-LSTM dynamics}) \quad (3)$$

$$\hat{z}_{t+1} = \text{JEPA}([z_t; g_t; a_t]) \quad (\text{predictor head}) \quad (4)$$

$$a_t = C(z_t, g_t, h_t) \quad (\text{controller}). \quad (5)$$

Enc_V produces $z_t \in \mathbb{R}^{256}$. The VAE term acts as a regulariser; the JEPA head predicts the *encoder output* z_{t+1} , not a reconstruction, and is trained with $\text{MSE}(z_{t+1}, \hat{z}_{t+1}) + \beta \text{KL}(q_\phi \| p)$. Surprise at step $t + 1$ is the predictor residual $\|\hat{z}_{t+1} - z_{t+1}\|_2$, which seeds curiosity bonuses and KG-enrichment triggers.

3.4 Active-inference LLM-fusion agent

The fused agent maintains a belief b_t over hidden state (opponent’s hand and library) using a particle filter constrained by the published decklist, and selects actions by minimising expected free energy:

$$\mathcal{F}(a) = \underbrace{-\mathbb{E}_{q(s'|a)}[R(s')]}_{\text{pragmatic}} - \underbrace{H[q(s'|a)] - \mathbb{E}[H[q(s'|s, a)]]}_{\text{epistemic (information gain)}} - \alpha \log p_{\text{LLM}}(a \mid \text{prompt}_t). \quad (6)$$

The LLM term provides a calibrated prior over the legal-action set; the controller of Section 3.3 provides R -estimates by short rollouts in the dream; and the KG provides hard combinatorial constraints (e.g., “no spell with mana cost > 3 in a deck whose curve peaks at 2”).

3.5 Self-play training loop

We use a champion-vs-challenger promotion scheme:

Algorithm 1 Champion-vs-Challenger Self-Play (Phase A.5).

- 1: Initialise pool $\mathcal{P}$ with K randomly-seeded agents.
 - 2: Pick champion $c^* \leftarrow \arg \max_{c \in \mathcal{P}} \text{ELO}(c)$.
 - 3: **for** round = 1, ..., R **do**
 - 4: Sample challenger $c \sim \mathcal{P} \setminus \{c^*\}$.
 - 5: Play N games c vs c^* , collect trajectories τ .
 - 6: Update ELO for both; train all pool members on τ .
 - 7: $w_r \leftarrow$ challenger win rate over the N games.
 - 8: **if** $w_r \geq \theta$ **then**
 - 9: Promote: $c^* \leftarrow c$; bump ELO by +50, demote former by −20.
 - 10: **end if**
 - 11: **end for**
-

θ is the promotion threshold (default 0.55). The loop is implemented in `src/training/rl_trainer.py` and is exercised end-to-end by `scripts/train_pipeline.py`.

4 Implementation

The codebase is organised into eight top-level packages under `src/`: `engine/` (Comprehensive-Rules engine), `knowledge/` (Neo4j, n10s, GraphRAG, GraphSAGE), `world_model/` (V, M, C, JEPA, KG-encoder, tokeniser), `agents/` (LLM, fusion, active-inference, opponent-model), `orchestrator/` (LangGraph runner, priority loop), `training/` (RL trainer, self-play, rewards, benchmarks), `judge/` (RAG over Comprehensive Rules) and `integrations/` (Scryfall, decklist loaders, SQLite cache). External services (Neo4j, Ollama) are orchestrated via `docker-compose`. The default LLM is `gemma4:e2b` (Gemma 4 edge variant) for low-end hardware compatibility, swappable to any Ollama-served model.

Reproducibility. All randomness flows through a single `numpy.random.Generator` seeded by `config.seed`. Decklists are committed alongside the code under `examples/`. The pipeline (`scripts/train_pipeline.py`) has seven stages (infra check, KG build, GNN embedding, self-play, JEPA, dream evaluation, model evaluation) each individually re-runnable.

5 Preliminary Evaluation

We report preliminary results; a full benchmark suite (Section 7) is in development. Current evidence is of three kinds.

(i) Engine correctness. The repository contains ~ 30 pytest modules covering combat, the stack, triggered/static/replacement abilities, and end-to-end games. The whole-suite runtime is dominated by integration tests; unit tests under `tests/test_engine/` and `tests/test_knowledge/` pass on Python 3.11 and 3.12.

(ii) Knowledge-graph coverage. The OWL ontology v1.1 declares 17 top-level classes and 24 object/data properties. After Scryfall ingestion of a representative Standard environment ($\sim 5,000$ cards) the graph contains the cards, their oracle-text SBERT embeddings, and combo edges imported from Commander Spellbook.

(iii) Self-play promotion loop. The promotion loop of Algorithm 1 is exercised by two unit tests (`tests/test_rl_trainer.py`); both pass. End-to-end promotion behaviour — whether champions stabilise after $\mathcal{O}(10)$ rounds of $N = 20$ games — is the next planned experiment and is not reported here.

We deliberately do not claim “state of the art” numbers: we know of no comparable public baseline that plays full-Comprehensive-Rules MTG, and the contribution of this report is the system itself, plus the ontology and the released self-play infrastructure.

6 Discussion and Limitations

Scope. The current engine targets a Standard subset and an extended Commander subset; banding, phasing, planeswalker loyalty abilities, and exotic targeting restrictions remain partial. The ontology v1.1 covers core mechanics but does not yet encode every keyword from every set — a known limitation that motivates the OntologyExtender workflow (Section 7).

Compute. Defaults are tuned for a single-GPU workstation: $z \in \mathbb{R}^{256}$, the LSTM hidden size is 512, the JEPA predictor is a 4-layer pre-norm Transformer, and the LLM is `gemma4:e2b`. Larger pretrained encoders or stronger LLMs (e.g., `llama-3.1-70b`) would likely improve play strength but were not the focus of this report.

Active inference vs. RL. Active inference here is implemented at the *decision* level (action selection conditional on world-model rollouts), not the *learning* level (no free-energy objective for the model itself). This is a deliberate choice: the world model is trained by the standard MSE+KL JEPA objective, which we found to be more compute-efficient than full Bayesian model-evidence maximisation.

LLM grounding. The LLM never produces game actions directly. It produces a posterior over the legal-action set provided by the engine; illegal actions are masked. This eliminates the dominant failure mode of LLM-only MTG bots (hallucinated cards, illegal plays) at the cost of a small expressivity loss.

7 Future Work

The following items are tracked in `IMPLEMENTATION_PLAN.md` and constitute the near-term roadmap:

1. **Strategy-aware and agent-driven mulligan policy.** The current keep/bottom heuristic is a single deterministic land-count rule shared by all agents. We are moving the decision through the agent interface (`MTGAgent.decide_mulligan`, `MTGAgent.bottom_cards`) so that aggressive, control, combo, and reactive strategies—and ultimately learned policies—can evaluate their own opening hands. JEPA-conditioned mulligan selection (rolling out a few imagined turns from each candidate hand) is the longer-term target.
2. **Priority-loop driven instants in the sync simulator.** The sync `GameSimulator` currently advances combat without exposing an instant-speed window to the non-active player; threading the existing `priority_loop` into combat steps will let counterspells and combat tricks fire mid-attack.
3. **Benchmark suite.** A `BenchmarkSuite` class with deterministic-replay seeds, four to six standard archetype decklists, baseline agents (Random, Heuristic, LLM-only, MCTS), and a per-game / per-agent metrics CSV.
4. **Ablation matrix.** Toggleable KG / dream / JEPA / opponent-model components with paired-game ELO comparison.
5. **Neural reasoner training.** A supervised + RL fine-tune of a graph-conditioned reasoner (`NeuralReasonerAgent`) replacing the SBERT card embedding with end-to-end GraphSAGE embeddings.
6. **Parallel self-play.** `asyncio.gather`-based parallel game execution to lift trajectory throughput, plus restoring `pytest-asyncio` so the async runner is exercised in CI.
7. **Belief-state visualisation.** JSON export and a Streamlit panel for the particle-filter posterior over the opponent’s hand.
8. **Commander politics.** Threat assessment across $N > 2$ opponents, including a multi-agent extension of the active-inference objective, and the Commander variant of the mulligan (free first mulligan).
9. **Ontology extension.** Continuous use of the OntologyExtender workflow (Sarker et al., 2024) to keep the OWL schema synchronised with each Standard rotation.

8 Conclusion

We have presented a research framework that approaches Magic: The Gathering not as “a harder Atari” but as a domain that fundamentally requires the combination of symbolic verification, structured knowledge, and learned imagination. The released codebase — engine, ontology, knowledge graph, V+M+C+JEPA world model, and active-inference LLM agent — is, to our knowledge, the first open system that puts these pieces together end-to-end on full Comprehensive-Rules MTG. We hope it is useful both as a benchmark for neuro-symbolic agents and as a sandbox for ideas in active inference, JEPA, and GraphRAG.

Reproducibility statement. All code, the OWL ontology, sample decklists, training scripts, and this report are available under MIT at <https://github.com/cuteredpwnda/opposition-agents-playing>

Acknowledgments

The authors thank the open-source maintainers of `open-mtg`, `Forge`, `XMage`, `Scrython`, `LangChain`, `LangGraph`, `Neo4j`, `n10s`, `APOC`, and `Ollama`, on whose work this project relies.

References

- D. Ha and J. Schmidhuber. World Models. *NeurIPS*, 2018.
- Y. LeCun. A Path Towards Autonomous Machine Intelligence. *OpenReview*, 2022.
- M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. *CVPR*, 2023.
- D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. *ICLR*, 2020.
- D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering Diverse Domains through World Models. *arXiv:2301.04104*, 2023.
- J. Schmidhuber. On Learning to Think: Algorithmic Information Theory for Novel Combinations of RL Controllers and Recurrent Neural World Models. *arXiv:1511.09249*, 2015.
- K. Irie et al. A Modern Self-Referential Weight Matrix that Learns to Modify Itself. *ICML*, 2022.
- K. Friston. The Free-Energy Principle: A Unified Brain Theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.
- N. Sajid, P. J. Ball, T. Parr, and K. J. Friston. Active Inference: Demystified and Compared. *Neural Computation*, 33(3):674–712, 2021.
- G. Tesauro. Temporal Difference Learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995.
- D. Silver et al. Mastering the Game of Go without Human Knowledge. *Nature*, 550(7676):354–359, 2017.
- D. Silver et al. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. *Science*, 362(6419):1140–1144, 2018.
- J. Schrittwieser et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, 588(7839):604–609, 2020.
- O. Vinyals et al. Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning. *Nature*, 575(7782):350–354, 2019.
- M. Moravčík et al. DeepStack: Expert-Level Artificial Intelligence in Heads-up No-Limit Poker. *Science*, 356(6337):508–513, 2017.
- N. Brown and T. Sandholm. Superhuman AI for Heads-up No-Limit Poker: Libratus Beats Top Professionals. *Science*, 359(6374):418–424, 2018.
- P. I. Cowling et al. Ensemble Determinization in Monte Carlo Tree Search for the Imperfect Information Card Game Magic: The Gathering. *IEEE Transactions on Computational Intelligence and AI in Games*, 2012.
- W. L. Hamilton, R. Ying, and J. Leskovec. Inductive Representation Learning on Large Graphs. *NeurIPS*, 2017.
- D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarisation. *arXiv:2404.16130*, 2024.
- N. F. Noy and D. L. McGuinness. Ontology Development 101: A Guide to Creating Your First Ontology. Stanford Knowledge Systems Laboratory, 2001.
- H. Knublauch and D. Kontokostas (eds.). Shapes Constraint Language (SHACL). *W3C Recommendation*, 2017.
- Neo4j Labs. Neosemantics (n10s): RDF and Semantics Toolkit for Neo4j. <https://neo4j.com/labs/neosemantics/>.
- M. K. Sarker et al. OntologyExtender: A Multi-Agent Human-in-the-Loop Framework for Ontology Evolution. *Technical Report*, 2024.